

Names: Norman Nomie, Drew Inglesby, Jacob Munly, Jack Dunlap

Class: CS362

Assignment: Project Proposal

– Textplorer –

Team Info

Team Members and Roles

Jacob Munly - Project Manager: In charge of creating the plan/schedule for the project, organizing the repository, assigning tasks, and ensuring deadlines are met.

Jack Dunlap - Documentation: Responsible for maintaining the project's documents, including repository documents and submission assignments and ensuring all criteria are met.

Drew Inglesby - UX/UI designer: Responsible for the conceptual layout of the game design, designing how users interact with the game, and the online aspect of the project.

Norman Nomie - Software Lead: Responsible for overseeing the software implementation, planning out the programming logic, and assigning pieces of code to team members.

Project Artifacts:

GitHub Repository Link: <https://github.com/gilbertboys/Textplorer>

Communication Plan:

We will be using both Discord and text messaging to communicate with each other while we work on this project. Discord is useful for being able to share files and images, and text messaging is great for communicating when we are away from our computers. We will communicate with each other whenever we feel stuck/confused, or one of us is making a major design choice/overhaul to our code. Beyond meeting with each other during each lecture, we will regularly schedule in-person meetings and remote meetings via Discord to discuss our project and work together. Discord works great for meeting remotely, as it supports screen sharing.

Product Description

Abstract: Textplorer is an innovative 2D platforming engine designed to simplify game design by bridging the gap between simple text editing and interactive level generation. Traditional level editors often rely on complex, nested menus that create a steep learning curve for casual creators. Textplorer bypasses this friction by using a coordinate-based parsing system to transform standard .txt files into playable environments in real-time. By leveraging the universal skill of typing, Textplorer offers a lightweight, accessible, and infinitely replayable experience where the user's vocabulary becomes the blueprint for their gameplay.

Goal: Our goal is to create a simple yet fun platforming game where users can easily create, share, and play levels. Games today often have massive file sizes and require powerful hardware, which leads to many people not being able to experience them. We want to create a lightweight and intuitive game that

Current Practice: Modern platforming games that include level editors often run into the issue of having giant nested menus where users can select from every object in the game. While this approach can work for die-hard fans of a game, it is often unintuitive for new users.

Novelty: What makes our approach to user generated platforming levels special, is that all users already possess the skill required to make a level. Everybody that owns a laptop or a computer knows how to type, which means that they can all create levels to play and share.

Effect: This project will impact users who enjoy simple video games or those who love the unique platformer experience. Our hope is for Textplorer to be a social game, where friends share fun levels that they have created or previously played. Textplorer offers a new way to create and play platforming challenges. The game allowing users to turn text into a playable map gives the game a near-infinite replayability, with the only limit being user creativity.

Technical Approach: The development of Textplorer focuses on creating a seamless pipeline between static data (text) and dynamic gameplay (physics). Our approach is divided into three core phases:

1. Data Parsing & Coordinate Mapping

The engine treats each .txt file as a two-dimensional coordinate grid. Using C++ file-stream libraries, the program parses the file character by character. Each character maps to a specific game object based on a predefined legend (e.g., | for a solid wall, _ for player start, ^ for hazards). This allows for precise level reconstruction where the text file's layout directly mirrors the in-game environment.

2. Physics & Collision Logic

Since the project moves beyond a Command Line Interface (CLI), we are implementing a custom physics engine to handle:

- AABB (Axis-Aligned Bounding Box) Collision: To detect interactions between the player and the environment blocks.

- Kinematics: Implementing gravity constants and velocity vectors to ensure movement feels fluid rather than rigid.

- State Management: Tracking whether a player is grounded, jumping, or interacting with a "goal" tile to trigger level completion.

3. Systems Architecture & UI

The application is built using a state-based architecture to manage transitions between the Main Menu, Level Loader, and Active Gameplay.

Level Manager: A module that scans the local directory for available .txt files and populates the UI list.

File I/O: An integrated uploader that validates user-created text files to ensure they contain necessary elements (like a start and end point) before allowing them to be played.

4. Development Environment

The project is authored in Visual Studio Code using C++. We utilize Git/GitHub for version control and a modular programming approach, allowing the Software Lead to assign specific components (like UI or Physics) to different team members simultaneously.

Risks: The most significant risk with developing our project is that our group doesn't have much experience developing games with graphics and physics. Most of our experience as developers so far has been with command line interfaces, which don't incorporate graphics or physics. We will minimize this risk by taking advantage of the plethora of beginner game development guides on YouTube.

Features

Major Features (MVP-Minimum Viable Product)

1. Physics & Movement: This feature is the core of the project, allowing users to control and interact with the levels within the game. The physics must be intuitive and consistent for users, and create exciting gameplay. Physics & movement includes jumping, gravity, moving left/right as well as how certain character blocks interact with the player.
2. Level Generation & Display: The program must be able to transform txt files from a "levels" folder into playable, interactive, real-time platforming levels.
3. File Upload: This feature allows users to upload txt files to the program, allowing users to create, share, and play their own levels.

4. User Interface: The user interface for a minimum viable product would include a main menu screen with options to Play, Upload, or Exit. Beyond that, the user would be able to pause while playing a level to bring up a pause screen that freezes the physics of a level and lets them either select Continue or Exit To Main Menu.

Stretch Goals:

1. Time Attack Gamemode: An alternate gamemode that a player can select from the main menu that adds a time limit to levels. If the timer reaches 0 during play, the user would fail the level and be forced to restart.
2. Visual Themes: The game would include selectable visual themes that would change the look of the game from a simple black on white color scheme to a variety of choices. An example of one of these visual themes would be a “Sky” theme, where the background is sky blue and the foreground is white like a cloud.